PRD — prd2pr Indexing Engine (Phase 1)

Nov 7, 2026 · @jayant sharma

1. Overview

Build the indexing engine of **prd2pr**, an open-source PRD-to-PR tool. Given a path to a local code folder, it parses every source file once, extracts symbols, chunks and relationships, and stores them in three searchable indexes: embeddings, a keyword index and a code graph. Later steps (planning from a PRD) query this index instead of reading the whole repo.

Goals

- One command, `prd2pr index`, builds a complete index of a TypeScript/JavaScript or Python repo.
- Re-running it only re-processes files that changed.
- Every symbol has one stable ID shared across PostgreSQL, Qdrant and the graph.
- Stores the index in PostgreSQL (one database can hold many repos) plus Qdrant; PostgreSQL starts with one docker compose command.

Non-goals: reading PRDs, planning, code generation, GitHub integration. Those are separate PRDs.

2. Scope

In scope	Out of scope
Walk a local folder, respect `.gitignore`	Cloning from GitHub (user clones first)
Languages: TypeScript, JavaScript, Python	Go, Java and others (plugin later)
Parse with tree-sitter, extract symbols, imports, calls, routes	Full type resolution via LSP/SCIP
PostgreSQL store (repos, files, symbols, chunks, edges), many repos in one database	User accounts, auth, web UI
Embeddings of code chunks and LLM summaries in Qdrant local mode	Hosted Qdrant, re-ranking models

In scope	Out of scope
Keyword search with PostgreSQL full-text search (tsvector + GIN)	Elasticsearch, OpenSearch
Import/call graph in PostgreSQL + PageRank repo map	Graph database (Neo4j)
Incremental re-index by content hash	File watching / live re-index
`prd2pr db migrate`, `init`, `index`, `status`, `search` (debug)	PRD parsing, plan, run

3. CLI commands and user flow

```
prd2pr db migrate                       # create/upgrade tables in PostgreSQL
prd2pr init   [--repo PATH]             # writes prd2pr.yaml, registers the
repo (repo_id)
prd2pr index  [--repo PATH] [--full] [--no-embed] [--no-summaries]
prd2pr status [--repo PATH]             # counts, last indexed commit, stale
files
prd2pr search "cancel order" [--k 10] [--mode keyword|semantic|graph|hybrid]
```

Flow

1. Start PostgreSQL with `docker compose up -d`, set `PRD2PR_DATABASE_URL`, and run `prd2pr db migrate` once.
2. Run `prd2pr init` once in a repo. It registers the repo in the `repos` table and gets a `repo_id`.
3. `prd2pr index` runs the full pipeline and shows a Rich progress bar per step.
4. Later runs of `prd2pr index` detect changed, new and deleted files and update only those. `--full` forces a rebuild.
5. `prd2pr search` lets the developer check retrieval quality by hand before the planning feature exists.

Index data lives in PostgreSQL (every table keyed by `repo_id`) and Qdrant (`.prd2pr/qdrant/`, local mode). Other local files in `<repo>/.prd2pr/`: `repo_map.md` and `logs/`.

4. Architecture

The pipeline runs in a fixed order. Steps 5a, 5b and 5c read from PostgreSQL and do not depend on each other, so they can run in parallel. The tree-sitter AST lives only in memory during step 3 and is never stored.

```
1. walk      folder path -> file list (filtered) + content hash
2. diff      compare hashes with the files table -> new / changed / deleted /
unchanged
3. parse     tree-sitter AST per new/changed file (in memory only)
4. extract   AST -> symbols + chunks + raw imports/calls  -> PostgreSQL
5a. embed    chunks (+ LLM summaries) -> vectors          -> Qdrant
5b. keyword  identifiers split -> tsvector column + GIN   -> PostgreSQL
5c. graph    resolve imports/calls -> edges               -> PostgreSQL
6. repo map  PageRank over graph -> .prd2pr/repo_map.md
7. commit    save last_indexed_commit + run stats on the repos row
```

Module layout

```
docker-compose.yml          # postgres:16 service for local development
src/prd2pr/
  cli.py                    # Typer app: db migrate, init, index, status,
search
  config.py                 # load/validate prd2pr.yaml (Pydantic)
  indexing/
    pipeline.py             # orchestrates steps 1-7, progress output
    walker.py               # step 1: walk + filter + hash
    differ.py               # step 2: change detection
    parser.py               # step 3: tree-sitter setup per language
    languages/
      base.py               # LanguageAdapter interface
      typescript.py         # queries for TS/JS
      python.py             # queries for Python
    extractor.py            # step 4: symbols, chunks, raw refs
    chunker.py              # chunk text + context header
    summarizer.py           # LLM summaries with hash cache
    embedder.py             # step 5a
    keyword.py              # step 5b: identifier splitting
    graph.py                # step 5c: name resolution + edges
    repo_map.py             # step 6
  storage/
    db.py                   # psycopg connection pool from
PRD2PR_DATABASE_URL
```

```
    migrations/
      0001_init.sql         # schema from section 6
    migrate.py              # applies pending .sql files, records
schema_migrations
    pg_store.py             # all PostgreSQL access behind one class
    vector_store.py         # Qdrant wrapper behind an interface
  llm/
    client.py               # LiteLLM wrapper (summaries, embeddings)
  search/
    retriever.py            # keyword, semantic, graph, hybrid (RRF)
tests/
  conftest.py               # spins up a throwaway Postgres DB per test
session
  fixtures/ts_sample/       # tiny TS repo with known symbols
  fixtures/py_sample/       # tiny Python repo with known symbols
```

Each language is a `LanguageAdapter` plugin, so adding a language means adding one file in `languages/` without touching the pipeline.

5. Functional requirements

FR1 — Walk and filter

- Recursively list files under the repo root, using paths relative to the root with `/` separators.
- Skip anything matched by `.gitignore` (use `pathspec`), plus a built-in list: `node_modules/`, `dist/`, `build/`, `.git/`, `.venv/`, `__pycache__/`, `*.min.js`, `*.lock`, `package-lock.json`, binaries, files over `max_file_kb` (default 500).
- Extra ignore globs come from `prd2pr.yaml`.
- Detect language by extension: `.ts .tsx .js .jsx .mjs .cjs` → typescript/javascript, `.py` → python. Other files are skipped and counted.
- Compute `sha256` of file bytes as `content_hash`.

FR2 — Change detection

- Compare each file's hash with the `files` table: **new** (not in table), **changed** (hash differs), **unchanged**, **deleted** (in table, not on disk).
- Only new and changed files go to FR3. Deleted files trigger cleanup (FR10).
- `--full` treats every file as changed.

FR3 — Parse

- Use `tree-sitter` with `tree-sitter-language-pack` (or per-language packages) to parse each file.
- A file with syntax errors still produces a tree. Record `parse_errors` count per file and continue; never stop the run.
- Do not persist the tree.

FR4 — Extract symbols, chunks and raw references

Each `LanguageAdapter` provides tree-sitter queries and returns:

Kind	TypeScript/JavaScript	Python
`class`	class declarations	class definitions
`function`	function declarations, exported arrow functions	top-level `def` / `async def`
`method`	class methods	methods inside classes
`route`	`router.get/post/put/patch/delete(...)`, NestJS `@Get/@Post/...` decorators	FastAPI/Flask decorators like `@app.post("/x")`, `@router.get`
`model`	Mongoose schemas, Prisma models if present, TypeORM `@Entity`	Pydantic `BaseModel`, SQLAlchemy models
`import`	`import ... from`, `require()`	`import x`, `from x import y`
`call`	call expressions inside each symbol	call expressions inside each symbol

For every symbol store: stable ID, name, qualified name (`Class.method`), kind, file, start/end line, signature (first line), docstring/JSDoc if present, parent symbol ID.

Chunks: one chunk per function/method/class body. Small symbols under `min_chunk_lines` (default 4) are merged into a file-level chunk. Symbols over `max_chunk_lines` (default 120) are split into parts on statement boundaries; each part keeps the same header. Header format:

```
File: src/orders/order.service.ts
Class: OrderService
Symbol: cancelOrder(id: string)
Imports: PaymentService, OrderRepo
```

Also extract string arguments of `process.env.X` / `os.getenv("X")` as `config_key` references.

FR5 — Summaries (optional, on by default)

- For each chunk, ask the configured cheap LLM for a 1–2 sentence plain-English summary of what the code does (business meaning, not syntax).
- Cache by `content_hash` of the chunk text: if unchanged, reuse the stored summary.
- Skip chunks under `min_summary_lines`. Batch requests, run with bounded concurrency (default 8), retry with backoff.
- `--no-summaries` disables this step.

FR6 — Embeddings

- Embed two texts per chunk: `header + code` → vector `code`; `summary` → vector `summary` (if a summary exists).
- Store in one Qdrant collection `chunks` with named vectors `code` and `summary`. Point ID = UUIDv5 of `"<repo_id>:<chunk_id>"`.
- Payload: `repo_id`, `chunk_id`, `symbol_id`, `file`, `kind`, `start_line`, `end_line`, `language`, `content_hash`. Code text is NOT stored in Qdrant; it lives in PostgreSQL.
- Every search filters on `repo_id`.
- Embedding model and dimension come from config and are saved on the `repos` row. Changing the model requires `--full`.

FR7 — Keyword index

- Each chunk row has an `identifiers` text column: symbol name, qualified name, path segments, and identifiers found in the code.
- Split identifiers: `cancelOrderHandler` → `cancel order handler`, `get_user_id` → `get user id`. Store both original and split forms.
- A generated `search_tsv` tsvector column (identifiers weight A, summary weight B, `simple` text search config) with a GIN index.

- Query with `websearch_to_tsquery('simple', :q)`, rank with `ts_rank_cd`, always filtered by `repo_id`.

FR8 — Code graph

- Edge types: `imports` (file→file), `calls` (symbol→symbol), `extends` (class→class), `handles_route` (route→handler symbol), `defines` (file→symbol), `uses_config` (symbol→config key).
- **Import resolution:** resolve relative paths, index files (`./orders` → `./orders/index.ts`), extensions, and `tsconfig.json` `paths` aliases (`@/x`). Python: package-relative and absolute module paths inside the repo. External packages become `external:<name>` nodes.
- **Call resolution:** for `Foo.bar()` or `bar()`, look up `Foo` / `bar` in the file's own symbols, then its imports. For `this.x.bar()` in a class, use the constructor parameter type of `x` if declared (NestJS DI). If unresolved, store the edge with `resolved = 0` and the raw name.

FR9 — Repo map

- Run PageRank (`networkx`) on the symbol graph.
- Write `.prd2pr/repo_map.md`: folder tree, then per file its top symbols by rank with signatures, until `repo_map_tokens` (default 4000) is reached.

FR10 — Incremental update and cleanup

- For a changed file: delete its old symbols, chunks, edges (where it is the source) and Qdrant points, then insert the new ones in one PostgreSQL transaction.
- For a deleted file: remove everything whose ID starts with `<file>::` and the file row.
- Edges from other files that pointed to removed symbols become `resolved = 0` and are re-resolved at the end of the run.
- Save `last_indexed_commit` (from `git rev-parse HEAD` if the folder is a git repo; otherwise null).

FR11 — Search (debug command)

- `keyword`: PostgreSQL full-text top-k (ts_rank_cd). `semantic`: Qdrant search on both named vectors, merge. `graph`: neighbors up to 2 hops of the top keyword hit.

- `hybrid`: reciprocal rank fusion `score = Σ 1/(60 + rank)` over keyword + semantic, then add 1-hop graph neighbors of the top 5.
- Output: rank, score, symbol, file:lines, which source found it.

6. Data model

Stable IDs (never random):

- Repo: `repos.id` (BIGSERIAL), looked up by absolute `root_path`. Every other table is keyed by `(repo_id, id)`.
- File: `src/orders/order.service.ts`
- Symbol: `src/orders/order.service.ts::OrderService.cancelOrder`
- Chunk: `<symbol_id>#<part>` (part = 0 unless split), file-level chunk `<file>::__file__#0`
- Qdrant point ID: `uuid5(NAMESPACE_URL, f"{repo_id}:{chunk_id}")`
- Overloads or duplicate names in one file: append `@<start_line>`

PostgreSQL schema (`storage/migrations/0001_init.sql`, PostgreSQL 16)

```
CREATE TABLE schema_migrations (
  version INTEGER PRIMARY KEY,
  applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE repos (
  id BIGSERIAL PRIMARY KEY,
  root_path TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL,
  last_indexed_commit TEXT,
  embed_model TEXT,
  embed_dim INTEGER,
  last_run_stats JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE files (
  repo_id BIGINT NOT NULL REFERENCES repos(id) ON DELETE CASCADE,
  path TEXT NOT NULL,
  language TEXT NOT NULL,
```

```
  content_hash TEXT NOT NULL,
  size_bytes INTEGER,
  parse_errors INTEGER NOT NULL DEFAULT 0,
  indexed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (repo_id, path)
);

CREATE TABLE symbols (
  repo_id BIGINT NOT NULL,
  id TEXT NOT NULL,
  file TEXT NOT NULL,
  name TEXT NOT NULL,
  qualified_name TEXT NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN
('class','function','method','route','model')),
  start_line INTEGER NOT NULL,
  end_line INTEGER NOT NULL,
  signature TEXT,
  docstring TEXT,
  parent_id TEXT,
  rank DOUBLE PRECISION NOT NULL DEFAULT 0,      -- PageRank score
  PRIMARY KEY (repo_id, id),
  FOREIGN KEY (repo_id, file) REFERENCES files(repo_id, path) ON DELETE
CASCADE
);

CREATE TABLE chunks (
  repo_id BIGINT NOT NULL,
  id TEXT NOT NULL,
  symbol_id TEXT,
  file TEXT NOT NULL,
  start_line INTEGER NOT NULL,
  end_line INTEGER NOT NULL,
  header TEXT NOT NULL,
  code TEXT NOT NULL,
  content_hash TEXT NOT NULL,
  summary TEXT,
  summary_hash TEXT,                             -- content_hash the summary
was made from
  embedded_hash TEXT,                            -- content_hash last
embedded
  identifiers TEXT NOT NULL DEFAULT '',          -- original + split
identifiers
```

```
  search_tsv TSVECTOR GENERATED ALWAYS AS (
    setweight(to_tsvector('simple', identifiers), 'A') ||
    setweight(to_tsvector('simple', coalesce(summary, '')), 'B')
  ) STORED,
  PRIMARY KEY (repo_id, id),
  FOREIGN KEY (repo_id, file) REFERENCES files(repo_id, path) ON DELETE
CASCADE
);

CREATE TABLE edges (
  repo_id BIGINT NOT NULL REFERENCES repos(id) ON DELETE CASCADE,
  src TEXT NOT NULL,
  dst TEXT NOT NULL,                              -- symbol/file id, external:
<pkg>, or raw name if unresolved
  type TEXT NOT NULL CHECK (type IN
('imports','calls','extends','handles_route','defines','uses_config')),
  src_file TEXT NOT NULL,
  resolved BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (repo_id, src, dst, type)
);

CREATE INDEX idx_symbols_file  ON symbols (repo_id, file);
CREATE INDEX idx_symbols_name  ON symbols (repo_id, name);
CREATE INDEX idx_chunks_file   ON chunks  (repo_id, file);
CREATE INDEX idx_chunks_search ON chunks  USING GIN (search_tsv);
CREATE INDEX idx_edges_dst     ON edges   (repo_id, dst);
CREATE INDEX idx_edges_srcfile ON edges   (repo_id, src_file);
```

Graph neighbors (used by `search --mode graph` and hybrid expansion):

```
WITH RECURSIVE nbr(id, depth) AS (
  SELECT %(start)s::text, 0
  UNION
  SELECT CASE WHEN e.src = n.id THEN e.dst ELSE e.src END, n.depth + 1
  FROM edges e
  JOIN nbr n ON e.src = n.id OR e.dst = n.id
  WHERE e.repo_id = %(repo_id)s AND e.resolved AND n.depth < %(hops)s
)
SELECT DISTINCT id FROM nbr;
```

Qdrant point (collection `chunks`)

```
{
  "id": "6f1c2c1e-…",
  "vector": { "code": [0.012, …], "summary": [-0.031, …] },
  "payload": {
    "repo_id": 1,
    "chunk_id": "src/orders/order.service.ts::OrderService.cancelOrder#0",
    "symbol_id": "src/orders/order.service.ts::OrderService.cancelOrder",
    "file": "src/orders/order.service.ts",
    "kind": "method",
    "start_line": 5,
    "end_line": 9,
    "language": "typescript",
    "content_hash": "a91f…"
  }
}
```

Create payload indexes on `repo_id`, `file`, `kind` and `language` for filtering.

7. Tech stack and config

Need	Choice
Language / packaging	Python 3.12, `uv`, `pyproject.toml`
CLI + output	Typer, Rich (progress bars, tables)
Config	Pydantic models, YAML via `pyyaml`
Parsing	`tree-sitter`, `tree-sitter-language-pack`
Ignore rules	`pathspec`
Database	PostgreSQL 16 (Docker for local dev)
DB driver	`psycopg[binary,pool]` (psycopg 3), plain SQL, no ORM
Migrations	numbered `.sql` files + small runner (`prd2pr db migrate`)
Keyword search	PostgreSQL full-text search (tsvector + GIN)
Vectors	`qdrant-client` in local mode (`path=.prd2pr/qdrant`)

Need	Choice
LLM + embeddings	`litellm` (any provider, incl. Ollama)
Graph / PageRank	edges in PostgreSQL, PageRank with `networkx`
Tests	`pytest`, a throwaway Postgres database per test session
Lint / types	`ruff`, `mypy`

`prd2pr.yaml`

```
version: 1
languages: [typescript, python]
ignore:
  - "migrations/**"
  - "**/*.generated.ts"
limits:
  max_file_kb: 500
  min_chunk_lines: 4
  max_chunk_lines: 120
  min_summary_lines: 6
database:
  url_env: PRD2PR_DATABASE_URL     # name of the env var holding the
connection URL
  pool_size: 5
llm:
  summary_model: "openai/gpt-4o-mini"        # any LiteLLM model id;
"ollama/<model>" for local
  embedding_model: "openai/text-embedding-3-small"
  embedding_dim: 1536
  concurrency: 8
summaries: true
repo_map_tokens: 4000
```

Secrets come only from environment variables (`PRD2PR_DATABASE_URL`, `OPENAI_API_KEY`, etc.), never from the YAML.

`docker-compose.yml` (local development)

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: prd2pr
      POSTGRES_PASSWORD: prd2pr
      POSTGRES_DB: prd2pr
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Matching URL: `postgresql://prd2pr:prd2pr@localhost:5432/prd2pr`.

8. Non-functional requirements

Performance

- Parse + extract + graph (no LLM) for a 1,000-file repo: under 60 seconds on a laptop with local PostgreSQL.
- Incremental run with fewer than 10 changed files and no LLM work: under 10 seconds.
- Write rows in batches (`executemany` or `COPY`), never one round trip per symbol.
- Memory: process files one at a time; never hold all ASTs at once.

Reliability

- One bad file (parse error, encoding error, huge file) is logged and skipped; the run continues.
- Every file's DB changes happen in one PostgreSQL transaction, so a crash never leaves half a file indexed.
- Two `prd2pr index` runs on the same repo must not overlap: take a PostgreSQL advisory lock on `repo_id` at start.
- LLM/embedding calls retry 3 times with exponential backoff; failed chunks are marked and retried on the next run.
- At start, check `schema_migrations`; if migrations are pending, stop and ask the user to run `prd2pr db migrate`.

- If the database is unreachable, fail fast with a clear message showing the host (never the password).

Privacy and safety

- No telemetry. Code leaves the machine only to the configured LLM/embedding endpoint, and not at all with Ollama or `--no-embed --no-summaries`.
- Skip files matching `.env*`, `*.pem`, `*.key`, `secrets.*` by default.

Observability

- Rich progress per step; final summary table (files, symbols, chunks, edges, unresolved edges, skipped files, LLM tokens, cost estimate, duration).
- Structured JSON log per run in `.prd2pr/logs/<timestamp>.jsonl`.

Code quality

- Type hints everywhere, `mypy --strict` on `indexing/` and `storage/`.
- No module outside `storage/` touches PostgreSQL or Qdrant directly.

9. Acceptance criteria and tests

Fixture repos (write these first, with a `expected.json` per fixture listing symbols and edges):

- `tests/fixtures/ts_sample/`: Express router, a NestJS-style controller + service with constructor injection, a model, a `tsconfig` path alias, an `index.ts` barrel, one file with a syntax error.
- `tests/fixtures/py_sample/`: FastAPI router, a service class, a Pydantic model, relative imports, a function using `os.getenv`.

Acceptance criteria

- [ ] `prd2pr db migrate` on an empty database creates all tables; running it again changes nothing.
- [ ] `prd2pr index` on each fixture produces exactly the symbols in `expected.json` (names, kinds, line ranges).
- [ ] At least 90% of call edges in fixtures are `resolved = true`; the rest are stored as unresolved, not dropped.
- [ ] Route `PATCH /orders/:id/cancel` has a `handles_route` edge to its controller method.

- [ ] The syntax-error file is indexed partially, counted in `parse_errors`, and the run exits 0.
- [ ] Running `prd2pr index` twice with no changes processes 0 files and makes 0 LLM calls.
- [ ] Editing one function changes only that file's rows and Qdrant points; IDs of untouched symbols stay identical.
- [ ] Deleting a file removes its symbols, chunks, edges and Qdrant points; `prd2pr status` shows the new counts.
- [ ] Indexing both fixtures into the same database keeps them apart: every query filtered by `repo_id` returns only that repo's rows.
- [ ] Summaries are reused when a chunk's code is unchanged (verified by mocked LLM call count).
- [ ] `prd2pr search "cancel order" --mode hybrid` returns `OrderService.cancelOrder` in the top 3 on the TS fixture.
- [ ] `--no-embed --no-summaries` works with no LLM API key set.
- [ ] `repo_map.md` exists and stays under the token limit.

Tests

- Unit: identifier splitting, ID generation, ignore rules, chunk splitting, import path resolution (incl. aliases).
- Integration: full index on both fixtures against a real PostgreSQL test database (created and dropped by `conftest.py`, URL from `PRD2PR_TEST_DATABASE_URL`), with LLM and embeddings mocked (deterministic fake vectors).
- One manual check on a real open-source repo of 500+ files, recording time and counts in the README.

10. Implementation milestones (build in this order)

Instruction for the coding agent: implement one milestone at a time, run its tests, and stop for review before starting the next. Do not implement anything listed as out of scope.

1. **M1 — Skeleton + database.** `pyproject.toml`, package layout from section 4, `docker-compose.yml`, `storage/db.py` (connection pool), `migrations/0001_init.sql`, `migrate.py`, Typer CLI with `db migrate` and stub commands, `config.py`, `prd2pr init` (registers the repo). Tests: config validation, migrations are idempotent.

2. **M2 — Walk + diff.** `walker.py`, `differ.py`, `pg_store.py` for `repos` and `files`. `prd2pr status` shows file counts. Tests: ignore rules, hashing, new/changed/deleted detection.
3. **M3 — Parse + extract (TypeScript first).** `parser.py`, `languages/typescript.py`, `extractor.py`, `chunker.py`, writes to `symbols` and `chunks`. Tests: TS fixture symbols match `expected.json`.
4. **M4 — Python adapter.** `languages/python.py`. Tests: Python fixture.
5. **M5 — Code graph.** Import and call resolution, `edges` table, `tsconfig` aliases, recursive-CTE neighbor query. Tests: edge expectations, resolved ratio.
6. **M6 — Keyword search.** `identifiers` column filled by `keyword.py`, `search_tsv` + GIN index, `prd2pr search --mode keyword`.
7. **M7 — Summaries + embeddings.** `llm/client.py`, `summarizer.py`, `embedder.py`, `vector_store.py`, hash-based caching. `--no-embed`, `--no-summaries`. Tests with mocked LLM.
8. **M8 — Hybrid search + repo map.** `retriever.py` (RRF + graph expansion), `repo_map.py` with PageRank.
9. **M9 — Incremental + cleanup.** FR10 end to end, advisory lock, re-resolution of dangling edges. Tests: edit/delete scenarios, two repos in one database.
10. **M10 — Polish.** Progress output, run summary table, JSON logs, README with a quickstart (docker compose → migrate → init → index) and measured numbers.

11. Open questions

- [ ] Default embedding model for users without an API key: ship an Ollama default, or require `--no-embed`?
- [ ] Should summaries run per symbol or per file for very large repos, to cut cost?
- [ ] Index React components and their props as their own `kind` now, or wait for the frontend PRD work?
- [ ] Now that PostgreSQL is required, move vectors to `pgvector` and drop Qdrant, so there is one store?
- [ ] Is `ts_rank_cd` good enough for keyword ranking, or add the ParadeDB `pg_search` extension for true BM25?